Virtual Machine in C

Mykyta 'Nick' Khomiakov

March 25, 2026

Contents

1 Abstract **2**

2 Project Setup **4**
2.1 Prerequisites 4
2.2 File Directory 5
2.3 Build 6
2.4 Running the VM 7
2.5 Separation of Concerns 8

3 Instruction Set Architecture **8**
3.1 Stack Manipulation 9
3.2 Arithmetic 10
3.3 Logic and Comparison 10
3.4 Control Flow 11
3.5 Registers 12
3.6 I/O and Control 13
3.7 Bytecode Encoding Summary 13

1 Abstract

This project presents the design and implementation of a stack-based virtual machine (VM) written entirely in C, conforming to the C11 standard. The machine supports a custom instruction set of 25 opcodes spanning five categories: arithmetic, logic and comparison, control flow, stack manipulation, and register operations. Alongside the VM core, the project includes a purpose-built lexer, a single-pass assembler, and an interactive Read-Evaluate-Print Loop (REPL) shell — all implemented from scratch with no external dependencies beyond the C standard library.

Motivation

The primary motivation for this project was a curiosity-driven investigation into how virtual machines, interpreters, and CPUs work at a fundamental level. Rather than treating a programming language runtime as a black box, the goal was to build one from the ground up — making explicit every design decision that is ordinarily hidden: how instructions are encoded, how operands are passed implicitly through a stack, how control flow is redirected, and how a call stack enables function abstraction.

Architecture Overview

The VM follows a classical *fetch–decode–execute* cycle. On each tick, the instruction pointer (`ip`) reads the next integer from a flat bytecode array (the *fetch* phase). The integer is dispatched through a `switch` statement (the *decode* phase), which routes execution to the appropriate handler function (the *execute* phase). The loop continues until a `HALT` instruction sets the running flag to zero.

The complete runtime state is encapsulated in a single `VM` struct:

- **Operand stack** (`int stack[256]`): The primary working memory. All arithmetic, logic, and comparison operations read their inputs from and write their results to this stack. The stack pointer `sp` always points to the next free slot.
- **General-purpose registers** (`int registers[16]`): Sixteen integer registers (R0–R15), accessible via the `LD` and `ST` instructions. Registers provide a simple means of persisting values across stack operations without re-pushing them every time.
- **Call stack** (`int call_stack[128]`): A dedicated return-address stack, separate from the operand stack, supporting up to 128 levels of nested function calls. The `CALL` instruction saves the current `ip` here before jumping; `RET` restores it.
- **Instruction pointer** (`ip`) and **call stack pointer** (`csp`): Integer indices into the bytecode array and call stack, respectively.

No global variables or hidden singletons exist anywhere in the codebase. Every function receives a pointer to the `VM` struct, making the state explicit and the code straightforward to reason about.

Why Stack-Based?

Most modern physical CPUs are register-based: they shuttle values between a fixed set of named registers (`eax`, `rbx`, `r0`, etc.) before operating on them. A stack-based VM instead uses an implicit operand convention — instructions always read from and write to the top of the stack, so no operand addresses need to be encoded in the instruction stream. This choice was made deliberately for the following reasons:

- **Implementation simplicity.** There is no register-allocation problem to solve. Every opcode knows exactly where its inputs are (the top of the stack) and where to leave its result (back on the top). The interpreter loop reduces to a clean `switch` over a flat integer array with no complex decode logic.
- **Compact bytecode.** Because operands are implicit, most instructions occupy a single integer word. Only instructions that require a literal value — `PUSH`, `JMP`, `JZ`, `JNZ`, `CALL`, `LD`, and `ST` — consume an additional word for that operand. This keeps the bytecode representation minimal and the assembler trivial to implement.
- **Portability.** The VM state is composed entirely of plain `int` arrays and integers. There are no platform-specific intrinsics, no inline assembly, and no OS-level system calls beyond `printf`. The same source compiles and behaves identically on Linux, macOS, and Windows via CMake.
- **Pedagogical clarity.** Stack machines have a direct correspondence with how arithmetic expressions are evaluated in postfix (Reverse Polish Notation) form. The execution model is easy to trace step by step, making this architecture a natural fit for studying how high-level expressions are lowered into a sequence of low-level operations — the same transformation performed by production compilers targeting the JVM or the .NET CLR.

The primary trade-off is instruction count: a stack machine often requires additional instructions (`DUP`, `SWAP`) to reorder operands that a register machine could address directly. This mirrors the philosophy of Reduced Instruction Set Computing (RISC), where a smaller, more uniform instruction set simplifies the processor at the cost of requiring more instructions to perform equivalent work. For a teaching and experimentation platform, this cost is entirely acceptable.

Toolchain: Lexer, Assembler, and REPL

Source text typed at the interactive prompt passes through three sequential stages before execution:

1. **Lexer** (`lexer.c`): Receives a raw line of text. Strips inline comments (any text following a `;` character), uppercases the first token to form the mnemonic, and parses up to two integer operands in either decimal or hexadecimal (`0x...`) notation. Produces a single `Token` struct.

2. **Assembler** (`assembler.c`): Looks up the mnemonic in a static table of 25 entries, each recording the instruction's opcode integer and expected operand count. Emits the opcode word followed by any operand words into a flat `Program` buffer (up to 1024 words). Issues a warning if the operand count does not match the table entry.

3. **VM** (`vm.c`): Receives the populated `Program` buffer, resets a fresh `VM` struct, and enters the fetch–decode–execute loop until `HALT` or a runtime error.

The REPL itself provides four shell commands — `RUN`, `LIST`, `RESET`, and `HELP` — that are intercepted before assembly and never enter the bytecode stream.

Error Handling

The VM detects and reports six categories of runtime error, printing a descriptive message and halting cleanly in each case: stack overflow (operand stack exceeds 256 slots), stack underflow (a pop or binary operation finds fewer values than required), division by zero (`DIV` with a zero divisor), invalid register index (`LD` or `ST` with an index outside 0–15), call stack overflow (`CALL` nested beyond 128 frames), and call stack underflow (`RET` with no matching `CALL`). In every case, the VM sets `running = 0` and returns normally — no crashes, no undefined behaviour.

Separation of Concerns

Each functional category of opcode lives in its own translation unit under `src/operations/`: `maths.c`, `logic.c`, `flow.c`, `registers.c`, and `stack.c`. The central dispatcher in `vm.c` contains only `case` labels; it delegates immediately to the relevant handler. Adding a new opcode requires touching at most three files: the `enum` in `vm.h`, the implementation file, and the mnemonic table in `assembler.c`. No other file needs to change. This modularity was a deliberate architectural goal, ensuring the codebase remains comprehensible as the instruction set grows.

2 Project Setup

2.1 Prerequisites

The project is built with **CMake 4.1** and targets the **C11** language standard. Before building, the following tools must be present on the host machine:

- **C compiler with C11 support** — either GCC (`gcc`) or Clang (`clang`) will work. The compiler must accept the `-std=c11` flag. GCC 5+ and Clang 3.3+ both satisfy this requirement.
- **CMake 4.1 or later** — used to generate the native build system files. CMake is platform-independent and produces Makefiles on Linux, Xcode project files on macOS, and Visual Studio solution files on Windows. Download from `cmake.org` or install via a package manager.
- **GNU Make** (Linux/macOS only) — required to consume the Makefile that CMake generates on Unix-like systems. On macOS, Make ships with the Xcode Command Line Tools (`xcode-select --install`). On Windows, CMake targets MSBuild or Ninja instead, so Make is not needed.

Recommended development environments by platform:

- **Windows** — Visual Studio 2022 (Community edition is free). CMake integrates natively and can target the MSVC toolchain or the bundled Clang front-end.
- **macOS / Linux** — JetBrains CLion, which has built-in CMake support and a bundled GDB/LLDB debugger. VS Code with the *CMake Tools* and *C/C++* extensions is a lightweight alternative.

2.2 File Directory

The repository is organised into three top-level directories: `src/` for all translation units, `include/` for their corresponding headers, and `docs/` for the LaTeX paper. Source files are further grouped by responsibility:

```
vm/
├── docs/
│   ├── sections/
│   │   ├── abstract.tex
│   │   └── project_setup.tex
│   │
│   ├── main.tex
│   └── main.pdf
│
├── src/
│   ├── main.c                # Interactive REPL entry point
│   ├── vm.c                  # Fetch-decode-execute loop; push/pop primitives
│   ├── io/
│   │   ├── lexer.c           # Tokenises a line of source text
│   │   └── assembler.c       # Translates tokens into bytecode words
│   └── operations/
│       ├── flow.c            # JMP, JZ, JNZ, CALL, RET
│       ├── logic.c           # EQ, LT, GT, AND, OR, NOT
│       ├── maths.c           # ADD, SUB, MULT, DIV, MOD
│       ├── registers.c       # LD, ST
│       └── stack.c           # PUSH, POP, DUP, SWAP, PEEK, PRINT, HALT
│
├── include/
│   ├── vm.h                  # VM struct, opcode enum, push/pop declarations
│   ├── io/
│   │   ├── lexer.h
│   │   └── assembler.h
│   └── operations/
│       ├── flow.h
│       ├── logic.h
│       ├── maths.h
│       ├── registers.h
│       └── stack.h
│
├── README.md
└── .gitignore
```

Figure 1: File Directory of the project

Every `.c` file under `src/operations/` is a self-contained translation unit. It includes only its own header and `vm.h`; it has no knowledge of sibling modules. This layout enforces at the filesystem level the same separation of concerns described in the architecture section.

2.3 Build

Option A — CMake (recommended)

CMake is the canonical way to build the project. From the repository root:

```
# 1. Create and enter a build directory
mkdir build && cd build

# 2. Configure (generates Makefile / solution)
cmake ..

# 3. Compile
cmake --build .
```

The resulting executable is placed at `build/virtual_machine` on Unix-like systems and `build\Debug\virtual_machine.exe` on Windows.

Option B — Direct GCC / Clang invocation

For environments without CMake, the project can be compiled with a single command from the repository root:

```
gcc -std=c11 -Wall -Wextra -Iinclude \
    src/main.c src/vm.c \
    src/io/lexer.c src/io/assembler.c \
    src/operations/maths.c src/operations/stack.c \
    src/operations/flow.c src/operations/logic.c \
    src/operations/registers.c \
    -o vm
```

Replace `gcc` with `clang` if that is the installed compiler. The flags `-Wall -Wextra` enable a broad set of compiler warnings; the project compiles cleanly at both levels.

2.4 Running the VM

Once compiled, launch the interactive assembler from the directory containing the binary:

```
./vm          # Unix-like systems
vm.exe        # Windows
```

The REPL greets the user and waits for input. A brief example session — pushing two values, adding them, printing the result, and halting — looks like this:

```
Virtual Machine -- interactive assembler
Type HELP for usage, QUIT to exit.

> PUSH 10
  [@ word 0]
> PUSH 32
  [@ word 2]
> ADD
  [@ word 4]
> PRINT
  [@ word 5]
> HALT
  [@ word 6]
>
--- Running (7 word(s)) ---
42
--- Done ---
```

The shell also accepts four meta-commands that are processed by the REPL and never assembled into the program:

Command	Effect
`RUN`	Assemble and execute the current buffer
`LIST`	Print each word with its bytecode index
`RESET`	Clear the program buffer
`HELP`	Display the full opcode reference
`QUIT` / `EXIT`	Exit the program

A blank line is equivalent to `RUN`. Lines beginning with `;` are treated as comments and stripped before lexing.

2.5 Separation of Concerns

The codebase is structured around a strict *separation of concerns*: no module knows about the internals of another module it does not directly depend on. The dependency graph is deliberately shallow:

- **Operation modules** (`maths`, `logic`, `flow`, `registers`, `stack`) depend only on `vm.h`. They are entirely unaware of one another.
- `vm.c` depends on all five operation headers, but only to dispatch via `case` labels. It contains no arithmetic or logic of its own.
- `assembler.c` depends on `vm.h` (for the opcode enum) and `lexer.h` (for the `Token` type). It knows nothing about runtime execution.
- `lexer.c` depends on nothing beyond the C standard library. It is the most isolated module in the project.
- `main.c` is the sole integration point, pulling together the lexer, assembler, and VM.

A practical consequence of this design: adding a new opcode requires changes to at most three files — the `enum` in `vm.h`, the relevant operation source file, and the mnemonic table in `assembler.c`. No other file needs to be touched, and the REPL automatically recognises the new instruction without any further modification.

3 Instruction Set Architecture

This section documents every opcode supported by the VM. Instructions are grouped into five categories matching the source layout: stack manipulation, arithmetic, logic and comparison, control flow, and registers. Each entry records the mnemonic, its stack effect using standard notation, a description of the operation, and any runtime error conditions it can trigger.

Notation

Stack effects are written in the form `before -- after`, where the rightmost element is the top of the stack. For example, `a b -- (a+b)` means two values are consumed and one is produced. The symbol `--` separates the state before the instruction executes from the state after. An empty left-hand side means the instruction pushes without consuming; an empty right-hand side means it consumes without pushing.

Opcodes that require a literal argument encoded in the bytecode stream carry it as an immediate word following the opcode. These are marked with `<n>` (an integer literal) or `<addr>` (a bytecode word index) in the mnemonic column.

3.1 Stack Manipulation

These instructions operate directly on the shape of the operand stack. They move, duplicate, or discard values without performing any computation on them.

Mnemonic	Stack Effect	Description
`PUSH <n>`	`-- n`	Push the integer literal `n` onto the stack.
`POP`	`a --`	Discard the top value.
`DUP`	`a -- a a`	Duplicate the top value; both copies remain.
`SWAP`	`a b -- b a`	Swap the top two values.
`PEEK`	`a -- a`	Print the top value without consuming it.

`PUSH` is the only way to introduce a literal integer into the program. Its operand is encoded as the word immediately following the opcode in the bytecode stream, so `PUSH 42` assembles to the two-word sequence `[PUSH, 42]`.

`DUP` is used when a value must both be tested (consumed by a comparison or jump) and retained for the next iteration of a loop. A typical countdown idiom reads:

```
DUP       ; keep a copy for the jump test
JZ  <end> ; consume the copy -- exit if zero
...
JMP <top> ; loop back to DUP
```

`SWAP` is necessary when the order of operands matters for a non-commutative operation. Because `b` is popped before `a`, the code sequence `PUSH 10 / PUSH 3 / SUB` computes $10 - 3$, not $3 - 10$. `SWAP` lets a program correct the order if operands were pushed in the wrong sequence.

`PEEK` is a diagnostic aid. It prints the top of the stack to standard output but leaves the stack unchanged, making it useful for observing intermediate values during development without disturbing the program's data.

Error conditions: `POP`, `DUP`, and `PEEK` halt with *stack underflow* if the stack is empty. `SWAP` halts with *stack underflow* if fewer than two values are present. `PUSH` halts with *stack overflow* if the stack already holds 256 values.

3.2 Arithmetic

Arithmetic instructions each consume two values from the top of the stack and push a single result. The value pushed second is treated as the left-hand operand and the value pushed first (i.e., the top) as the right-hand operand, consistent with the natural push order.

Mnemonic	Stack Effect	Description
`ADD`	`a b -- (a+b)`	Integer addition.
`SUB`	`a b -- (a-b)`	Integer subtraction.
`MULT`	`a b -- (a*b)`	Integer multiplication.
`DIV`	`a b -- (a/b)`	Integer division (truncates toward zero).
`MOD`	`a b -- (a%b)`	Remainder after integer division.

All five instructions operate on signed 32-bit integers (the C `int` type on all target platforms). Overflow wraps silently under standard C integer arithmetic rules; the VM does not detect it.

`DIV` performs truncating integer division: $7 \div 2 = 3$, and $-7 \div 2 = -3$. The result is always rounded toward zero, matching C's built-in `/` operator for signed integers.

`MOD` computes the remainder consistent with `DIV`: the sign of the result matches the sign of the dividend, so $-7 \% 2 = -1$.

Error conditions: All five instructions halt with *stack underflow* if fewer than two values are present. `DIV` additionally halts with *division by zero* if the divisor (top of stack) is zero before the pop.

3.3 Logic and Comparison

Comparison instructions treat any non-zero integer as *true* and zero as *false*. They consume their operands and push either `1` (true) or `0` (false), allowing their results to be used directly as conditions for the conditional jump instructions `JZ` and `JNZ`.

Mnemonic	Stack Effect	Description
`EQ`	`a b -- (a==b)`	Push 1 if `a` equals `b`, else 0.
`LT`	`a b -- (a<b)`	Push 1 if `a` is less than `b`, else 0.
`GT`	`a b -- (a>b)`	Push 1 if `a` is greater than `b`, else 0.
`AND`	`a b -- (a&&b)`	Push 1 if both `a` and `b` are non-zero, else 0.
`OR`	`a b -- (a||b)`	Push 1 if either `a` or `b` is non-zero, else 0.
`NOT`	`a -- (!a)`	Push 1 if `a` is zero; push 0 otherwise.

`EQ`, `LT`, and `GT` provide the full set of binary comparisons needed to express any relational condition. Less-than-or-equal ($a \leq b$) can be expressed as `GT NOT` (negate the greater-than result); greater-than-or-equal as `LT NOT`; not-equal as `EQ NOT`.

AND and OR implement *logical* (not bitwise) conjunction and disjunction. Any non-zero value is treated as true, so AND of 5 and 3 yields 1, not 1 (bitwise). This matches the semantics of C's && and || operators.

A typical guarded conditional pattern — execute a block only if two conditions both hold — looks like:

```
; stack: ... a b
AND           ; stack: ... (a && b)
JZ  <skip>    ; jump past the block if false
... ; body
<skip>:
```

Error conditions: All binary instructions (EQ, LT, GT, AND, OR) halt with *stack underflow* if fewer than two values are present. NOT halts with *stack underflow* if the stack is empty.

3.4 Control Flow

Control flow instructions alter the instruction pointer (ip), enabling loops, conditionals, and function calls. All jump targets are expressed as absolute word indices into the bytecode array — the same indices reported by the LIST shell command.

Mnemonic	Stack Effect	Description
JMP <addr>	--	Unconditional jump to word index addr.
JZ <addr>	a --	Pop a; jump to addr if a is zero.
JNZ <addr>	a --	Pop a; jump to addr if a is non-zero.
CALL <addr>	--	Push return address to call stack; jump to addr.
RET	--	Pop return address from call stack; resume there.

JMP sets ip to the given word index unconditionally. Because the assembler emits each instruction's opcode at a known word offset (reported by [@ word N] in the REPL), jump targets can be calculated by inspection. Note that JMP itself occupies two words (opcode + operand), so a backward jump to the instruction at word 4 requires the operand 4, not an offset.

JZ and JNZ are the conditional branches. They always consume the top of the stack regardless of whether the jump is taken. This means the condition value must be produced fresh on each iteration of a loop — typically via DUP before a comparison — if the underlying counter or flag must be preserved.

CALL saves the address of the instruction *immediately following* the CALL opcode word and its operand (i.e., ip after both words have been consumed) onto the call stack, then sets ip to the target address. RET restores ip from the top of the call stack, resuming execution at the saved address. This mechanism supports up to 128 levels of nested calls before the call stack overflows.

Parameters are conventionally passed on the operand stack, pushed by the caller before CALL and consumed by the function body. Return values are left on

the operand stack by the function before RET. This convention is not enforced by the hardware; it is a programming discipline.

An example of a simple function call that doubles its argument:

```
; --- caller ---
PUSH 7        ; word 0: push argument
CALL 6        ; word 2: call function at word 6; return
    address = word 4
PRINT         ; word 4: print the return value
HALT          ; word 5

; --- function body (starts at word 6) ---
DUP           ; word 6: duplicate argument (a a)
ADD           ; word 7: a + a = 2a
RET           ; word 8: return to word 4
```

Output: 14

Error conditions: JZ and JNZ halt with *stack underflow* if the stack is empty. CALL halts with *call stack overflow* if 128 frames are already active. RET halts with *call stack underflow* if there is no matching CALL on the call stack. No bounds checking is performed on jump targets; jumping to an out-of-range address produces undefined behaviour.

3.5 Registers

The VM provides sixteen general-purpose integer registers, addressed by index 0 through 15. Registers hold their values across instructions and are initialised to zero at VM startup. They offer a way to store and retrieve values without keeping them permanently on the operand stack.

Mnemonic	Stack Effect	Description
ST <reg>	a --	Pop a and store it in register reg.
LD <reg>	-- a	Push the current value of register reg.

ST is a destructive write: the value is removed from the operand stack and placed in the register. LD is non-destructive with respect to the register: the register retains its value after the push. A register can be read any number of times with repeated LD instructions.

Registers are particularly useful for loop counters and function arguments that must survive multiple stack operations. Rather than duplicating a value repeatedly across loop iterations, it can be stored once in a register and reloaded as needed:

```
PUSH 10
ST 0        ; register 0 = 10 (loop limit)
PUSH 1
ST 1        ; register 1 = 1  (counter, starts at 1)
```

```
; loop body (word 8)
LD 1        ; push counter
PRINT       ; print it
LD 1        ; push counter again
PUSH 1
ADD         ; counter + 1
ST 1        ; store updated counter
LD 1        ; push counter for comparison
LD 0        ; push limit
GT          ; counter > limit?
JNZ 8      ; if not, repeat
HALT
```

Output: `1`, `2`, ..., `10`

Error conditions: Both `ST` and `LD` halt with *invalid register* if the register index is outside the range 0–15. `ST` additionally halts with *stack underflow* if the stack is empty when the store is attempted.

3.6 I/O and Control

Mnemonic	Stack Effect	Description
`PRINT`	`a --`	Pop the top value and print it to standard output.
`HALT`	`--`	Stop execution; set the running flag to zero.

`PRINT` is the VM's sole output instruction. It pops the top value, formats it as a signed decimal integer, and writes it to `stdout` followed by a newline. It is intentionally destructive: if the value must be retained, `DUP` should precede it.

`HALT` terminates the fetch–decode–execute loop cleanly. Every well-formed program must end with `HALT`; without it, the VM will continue reading beyond the end of the bytecode array, causing undefined behaviour.

3.7 Bytecode Encoding Summary

The table below summarises the word width of each instruction as it appears in the flat bytecode array. Instructions with an operand occupy two words; all others occupy one.

Instruction(s)	Words	Operand
`ADD, SUB, MULT, DIV, MOD`	1	—
`EQ, LT, GT, AND, OR, NOT`	1	—
`POP, DUP, SWAP, PEEK`	1	—
`PRINT, HALT, RET`	1	—
`PUSH <n>`	2	Integer literal
`JMP <addr>, JZ <addr>, JNZ <addr>`	2	Word index
`CALL <addr>`	2	Word index
`LD <reg>, ST <reg>`	2	Register index (0–15)

Because all values are plain C `int`s, opcodes and operands share the same representation. The distinction between an opcode word and a data word is purely positional: the VM knows from context (specifically, from the `operand_count` field in the assembler's mnemonic table) how many words following an opcode are operands rather than the next opcode.